

INFORME FINAL – GESTION DE BASE DE DATOS HARDWARESHOP

PRESENTADO POR:
JHAN BRANDON RIASCOS

PRESENTADO A:
ING. BRAYAN ARCOS

INSTITUTO TECNOLOGICO DEL PUTUMAYO
TECNOLOGIA EN DESARROLLO DE SOFTWARE
DESARROLLO DE BASE DE DATOS
MOCOA
2024

Tabla de contenido

1.	Introducción.....	3
1.1	Alcance del Informe.....	3
1.2	Objetivo del Proyecto	4
1.3	Objetivos específicos del proyecto	4
2	Metodología.....	4
2.1	Uso de herramientas	5
3	Desarrollo del Informe	5
3.1	Descripción del negocio y el Proyecto de desarrollo: HardwareShop.....	5
3.2	Descripción de tablas.....	6
3.3	Normalización de la Base de Datos	8
3.4	Tipos de relaciones	10
3.5	Funciones básicas de SQL usada en la herramienta	11
3.5.1	Manipulación de caracteres:	11
3.5.2	Uso de funciones matemáticas.....	13
3.5.3	Ejemplos de uso en la herramienta	13
3.5.4	Funciones agregadas.....	14
3.6	Subconsultas en SQL.....	18
3.6.1	Subconsulta en el WHERE	18
3.6.2	Subconsulta en el FROM	20
3.6.3	Subconsulta en el SELECT	21
3.7	Vistas.....	22
3.8	Triggers	24
3.9	Procedimientos Almacenados (Stored Procedures)	25
3.10	Encriptación de Contraseñas.....	28
3.11	Respaldo de Bases de Datos.....	30
4	Conclusiones	31
5	Recomendaciones	31

1. Introducción

Este informe tiene como propósito documentar el diseño, desarrollo e implementación de una base de datos relacional para una ferretería ficticia llamada hardwareshop. La motivación principal para realizar este proyecto radica en la necesidad de contar con un sistema robusto que permita gestionar de manera eficiente las operaciones de una ferretería física, incluyendo la administración de productos, inventarios, ventas y usuarios. El informe busca plasmar no solo el proceso técnico, sino también las mejores prácticas empleadas para garantizar que la base de datos sea escalable, segura y alineada con los estándares actuales de la industria.

A continuación, adjunto el link del trabajo con el diseño del modelo ER, los scripts con la creación de la documentación, los scripts de consultas, creación de triggers, creación de vistas, creación de procesos almacenados, e instrucciones para el encriptado de información.

<https://github.com/jhanbrandon/BaseDeDatosLocal>

1.1 Alcance del Informe

Este informe abarca el diseño y desarrollo completo de la base de datos, desde la identificación de las necesidades del negocio hasta la implementación de soluciones técnicas específicas. Incluye:

Diseño de la Base de Datos:

- Normalización de tablas.
- Definición de relaciones entre entidades.
- Estructura general y nomenclatura estándar.

Implementación de Funcionalidades:

- Automatización de procesos mediante triggers y procedimientos almacenados.
- Validación de datos para garantizar la integridad.
- Gestión de seguridad, incluyendo encriptación de contraseñas.

Gestión de Respaldo:

- Configuración y uso de herramientas como mysqldump y MySQL Workbench para la exportación y restauración de datos.

Casos de Uso y Consultas:

- Ejemplos prácticos de consultas SQL, subconsultas y optimización.

El alcance se limita al desarrollo del modelo lógico y físico de la base de datos y su implementación en un entorno MySQL. No incluye el desarrollo de aplicaciones cliente o interfaces gráficas para interactuar con la base de datos.

1.2 Objetivo del Proyecto

Diseñar e implementar una base de datos relacional para una ferretería ficticia, que permita gestionar eficientemente las operaciones comerciales y administrativas mediante un sistema seguro, escalable y automatizado.

1.3 Objetivos específicos del proyecto

- Modelar un esquema de base de datos normalizado que represente las operaciones principales de la ferretería.
- Implementar mecanismos de seguridad, como encriptación de contraseñas, para proteger datos sensibles.
- Automatizar tareas repetitivas y cálculos mediante triggers y procedimientos almacenados.
- Diseñar un sistema de respaldo que garantice la disponibilidad y recuperación de datos en caso de fallos.
- Documentar el proceso técnico de diseño e implementación para asegurar la reproducibilidad y comprensión del sistema.

2 Metodología

El enfoque metodológico utilizado fue la coordinación por parte del docente donde en cada clase se expuso el tema a abordar y se dejaron consultas. Semanalmente el docente hace revisión de las bases de datos de los estudiantes y resuelve inquietudes respecto al diseño de la base de datos haciendo apreciaciones basadas en su experiencia.

A continuación, los pasos que se siguieron:

1. Análisis de Requerimientos:

- Identificación de las necesidades del negocio, como gestión de productos, usuarios, roles, ventas e inventarios.
- Definición de funcionalidades clave como encriptación de contraseñas, automatización mediante triggers y respaldo de datos.

2. Diseño:

- Creación del modelo lógico y físico de la base de datos, asegurando una estructura normalizada y eficiente.
- Diseño de diagramas de entidad-relación (ER) para visualizar las relaciones entre tablas.

3. Implementación:

- Desarrollo de la base de datos utilizando MySQL.
- Automatización de procesos mediante triggers y procedimientos almacenados.
- Configuración de mecanismos de respaldo y seguridad.

4. Pruebas:

- Validación de consultas SQL y procedimientos almacenados para garantizar la integridad y consistencia de los datos.
- Simulación de casos de uso, como creación de facturas y gestión de inventarios, para verificar el funcionamiento del sistema.

5. Documentación:

- Registro detallado del diseño, implementación y configuración de herramientas.

- Preparación de este informe como evidencia del proceso y guía para futuras mejoras.

2.1 Uso de herramientas

El uso de herramientas específicas facilitó cada etapa del proyecto, desde el diseño inicial hasta la implementación y pruebas:

MySQL Workbench: Herramienta principal para la creación y gestión de la base de datos, entre sus funcionalidades se utiliza para diseñar y visualizar el esquema de la base de datos mediante su editor gráfico, además de generar respaldos utilizando la funcionalidad de Data Export.

GitHub: Plataforma para el control de versiones del proyecto, facilitó el registro de cambios y colaboraciones, asegurando la trazabilidad y protección del código fuente y los scripts SQL.

Laragon: Servidor local utilizado para ejecutar MySQL y realizar pruebas en un entorno controlado, permitió simular el entorno de producción durante las fases de implementación y pruebas.

Microsoft Visio: Herramienta utilizada para la creación de diagramas de flujo de trabajo y diseño de procesos relacionados con el sistema.

Draw.io: Utilizada para diseñar los diagramas de entidad-relación (ER) de la base de datos. Permitted una visualización clara de las relaciones y dependencias entre tablas.

3 Desarrollo del Informe

3.1 Descripción del negocio y el Proyecto de desarrollo: HardwareShop

Líneas de producto

La ferretería es una empresa que ofrece 4 líneas de productos:

1. **Herramientas:** Martillos, taladros, destornilladores y otros productos manuales y eléctricos.
2. **Materiales de Construcción:** Cemento, ladrillos, arena, madera, etc.
3. **Pinturas y Accesorios:** Pinturas de diferentes colores, brochas, rodillos, entre otros.
4. **Electricidad y Fontanería:** Cableado eléctrico, enchufes, tubos PVC, etc.

Sedes y Ubicación:

1. **Sede Principal (Putumayo):** Localizada estratégicamente en la zona para atender proyectos de construcción locales.
2. **Sucursal (Huila):** Ubicada en un centro urbano para atender un mercado más amplio.

Roles Existentes:

1. **Administrador:** Encargado de la gestión general, incluyendo la supervisión de las operaciones y la toma de decisiones estratégicas.
2. **Empleado:** Responsable de las ventas y la gestión del inventario.
3. **Cliente:** Representa a los usuarios que realizan compras.
4. **Proveedor:** Suministra materiales y productos para la ferretería.

Necesidades específicas de la Ferretería:

1. **Gestión de Medios de Pago:** Incorporar métodos como pagos en efectivo, transferencias bancarias, y plataformas electrónicas como Nequi o Daviplata.
2. **Registro de Clientes y Proveedores:** Facilitar el seguimiento de transacciones y optimizar la logística.
3. **Administración de Usuarios por Roles:** Asegurar que cada usuario tenga acceso restringido según sus funciones.
4. **Medidas de Seguridad en Datos:** Uso de contraseñas seguras con almacenamiento encriptado. Activadores (Triggers) para validar datos ingresados y prevenir errores como precios o stocks negativos.
5. **Historial de Datos Importantes:** Registro de cambios en precios y roles para auditorías.
6. Control de movimientos de inventarios y facturación.
7. **Seguridad en la Base de Datos:** Asignación de roles a usuarios con privilegios limitados según sus funciones.

Necesidades específicas del equipo de desarrollo

1. Usuarios del equipo de desarrollo con acceso restringido a tablas críticas.
2. Medidas de respaldo a la totalidad de los datos de forma regular para garantizar la recuperación de datos en caso de fallos.
3. Almacenamiento de respaldos en ubicaciones seguras y externas, como la nube.

3.2 Descripción de tablas

Nombre de la Tabla	Propósito de la Tabla	Campos de la Tabla	Campos Clave
--------------------	-----------------------	--------------------	--------------

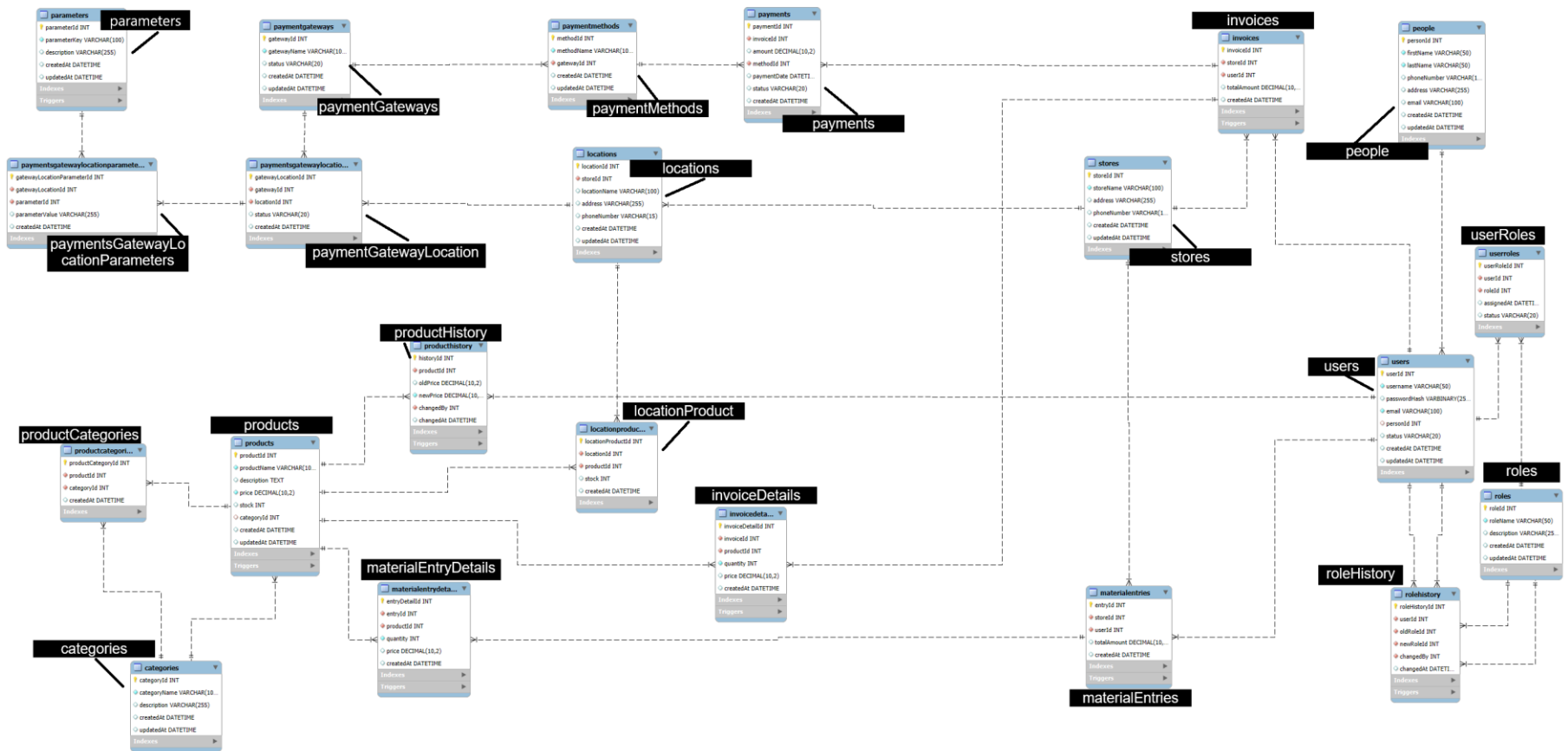
products	Gestionar los datos de los productos, como su precio, descripción y stock.	- productId - productName - description - price - stock - categoryId - createdAt - updatedAt	- productId: Identifica de forma única cada producto.
categories	Clasificar los productos por tipo o categoría.	- categoryId - categoryName - description - createdAt - updatedAt	- categoryId: Identifica de forma única cada categoría.
users	Almacenar información de los usuarios y sus credenciales encriptadas.	- userId - username - password - email - personId - status - createdAt - updatedAt	- userId: Identifica de forma única a cada usuario.
roles	Definir los roles y permisos de los usuarios.	- roleId - roleName - description - createdAt - updatedAt	- roleId: Identifica de forma única cada rol.
invoices	Registrar las facturas generadas por ventas.	- invoiceId - userId - totalAmount - createdAt - updatedAt	- invoiceId: Identifica de forma única cada factura.
invoiceDetails	Registrar los detalles de los productos vendidos en cada factura.	- invoiceDetailId - invoiceId - productId - quantity - price - createdAt	- invoiceDetailId: Relaciona cada detalle con una factura específica.

materialEntries	Registrar las entradas de material al inventario.	- entryId - userId - totalAmount - createdAt - updatedAt	- entryId: Relaciona una entrada con sus detalles.
materialEntryDetails	Registrar los detalles de cada entrada de material.	- entryDetailId - entryId - productId - quantity - price - createdAt	- entryDetailId: Relaciona detalles con una entrada específica.
productHistory	Mantener un historial de cambios en los productos.	- historyId - productId - oldPrice - newPrice - changedBy - changedAt	- historyId: Identifica cambios únicos en los productos.
roleHistory	Mantener un historial de cambios en los roles asignados a usuarios.	- roleHistoryId - userId - oldRoleId - newRoleId - changedBy - changedAt	- roleHistoryId: Identifica los cambios únicos de roles de usuarios.

La base de datos diseñada para este proyecto ha sido redactada en inglés, siguiendo estándares internacionales en desarrollo de software. Se ha utilizado el modelo camelCase para los nombres de tablas y campos, garantizando consistencia y legibilidad. Por ejemplo, nombres como productId y createdAt reflejan esta convención, permitiendo que desarrolladores de distintos contextos puedan trabajar con la base de datos sin problemas de interpretación.

3.3 Normalización de la Base de Datos

La base de datos ha sido normalizada, siguiendo las mejores prácticas de diseño relacional. La normalización es un proceso que organiza las tablas y sus relaciones para minimizar redundancias y dependencias no deseadas, mejorando la integridad y eficiencia de los datos.



3.4 Tipos de relaciones

Relación uno a uno

Una relación uno a uno significa que cada registro en una tabla está relacionado con exactamente un registro en otra tabla.

- **users ↔ people**

Cada usuario (userId) tiene exactamente una información personal (personId).

Esto permite almacenar información personal (nombre, dirección, etc.) de manera separada de las credenciales del usuario. Esto es útil para mantener la seguridad y cumplir con principios de separación de datos.

Relación uno a muchos

En una relación uno a muchos, un registro de una tabla puede estar relacionado con varios registros de otra tabla. Esto es ideal para representar jerarquías o listas asociadas.

Estas relaciones son comunes en sistemas relacionales, ya que reflejan jerarquías naturales (como categorías y productos) y registros transaccionales (como facturas y sus detalles).

- **categories ↔ products:**

Una categoría puede tener varios productos relacionados, pero cada producto pertenece a una sola categoría.

- **users ↔ invoices:**

Un usuario (por ejemplo, un vendedor) puede generar varias facturas, pero cada factura está asociada a un único usuario.

- **invoices ↔ invoiceDetails:**

Una factura puede tener múltiples productos vendidos (detalles de factura), pero cada detalle pertenece a una sola factura.

- **materialEntries ↔ materialEntryDetails:**

Una entrada de material puede contener varios detalles (productos específicos), pero cada detalle pertenece a una única entrada.

Relaciones Muchos a Muchos

En una relación muchos a muchos, varios registros de una tabla pueden estar relacionados con varios registros de otra tabla. Estas relaciones generalmente se implementan utilizando tablas intermedias.

- **users ↔ roles (a través de userRoles):**

Un usuario puede tener múltiples roles (por ejemplo, "Administrador" y "Empleado"), y un rol puede ser asignado a múltiples usuarios. Permite la administración flexible de permisos, adaptándose a diferentes necesidades de acceso.

- **products ↔ categories (a través de productCategories):**

Un producto puede pertenecer a múltiples categorías, y una categoría puede contener múltiples productos. Algunos productos pueden encajar en más de una categoría, como una herramienta eléctrica que también puede clasificarse como equipo de construcción.

3.5 Funciones básicas de SQL usada en la herramienta

Es importante que los usuarios de la ferretería conozcan la potencia que tiene SQL a nivel de reportes es por eso que se comparte con ellos algunas de las funcionalidades:

3.5.1 Manipulación de caracteres:

- **CONCAT:** Permite combinar o unir valores de diferentes columnas o textos en una sola cadena.
- **LENGTH:** Devuelve la longitud de una cadena, es decir, el número de caracteres que contiene.
- **UPPER:** Convierte una cadena de texto a mayúsculas.
- **LOWER:** Convierte una cadena de texto a minúsculas.
- **SUBSTRING:** Extrae una parte específica de una cadena, según la posición inicial y la longitud indicada.

EJEMPLOS DE USO EN LA HERRAMIENTA

-- **CONCAT:** Combina el nombre y el apellido de las personas en la tabla people

```
SELECT CONCAT(firstName, ' ', lastName) AS fullName  
FROM people;
```

	fullName
►	Carlos Pérez
	María Gómez
	Luis Rojas
	Ana Martínez
	Pedro Ramírez
	Sofía López
	Camilo Torres
	Diana Muñoz
	Miguel Castro

-- **LENGTH:** Obtiene la longitud del nombre de cada categoría en la tabla categories

```
SELECT categoryName, LENGTH(categoryName) AS nameLength  
FROM categories;
```

	categoryName	nameLength
▶	Herramientas	12
	Materiales de construcción	27
	Pinturas	8

-- UPPER: Convierte los nombres de los métodos de pago a mayúsculas

```
SELECT methodName, UPPER(methodName) AS methodNameUpperCase
FROM paymentMethods;
```

	methodName	methodNameUpperCase
▶	Transferencia Bancaria	TRANSFERENCIA BANCARIA
	Pago con Nequi	PAGO CON NEQUI
	Pago con Daviplata	PAGO CON DAVIPLATA

-- LOWER: Convierte los nombres de las tiendas a minúsculas

```
SELECT storeName, LOWER(storeName) AS storeNameLowerCase
FROM stores;
```

	storeName	storeNameLowerCase
▶	Ferretería Putumayo	ferretería putumayo
	Ferretería Huila	ferretería huila

-- SUBSTRING: Extrae los primeros 5 caracteres del nombre de cada producto en la tabla products

```
SELECT productName, SUBSTRING(productName, 1, 5) AS productShortName
FROM products;
```

	productName	productShortName
▶	Cemento Gris	Cemen
	Ladrillo	Ladri
	Martillo	Marti
	Pintura Blanca	Pintu
	Rodillo	Rodil
	Taladro	Talad
	Taladro	Talad
	Taladro	Talad

-- SUBSTRING Extraer los primeros 5 caracteres de los nombres de usuario

```
SELECT
    username,
    SUBSTRING(username, 1, 5) AS shortUsername
FROM users;
```

	username	shortUsername
►	anam	anam
	camilot	camil
	carlosp	carlo
	dianam	diana
	luisr	luisr
	mariag	maria
	miguelc	migue
	pedror	pedro
	sofial	sofia

3.5.2 Uso de funciones matemáticas

- **ROUND:** Redondea un número a un número específico de decimales, útil para ajustar valores en reportes financieros o métricas precisas.
- **ABS:** Devuelve el valor absoluto de un número, eliminando su signo negativo, ideal para analizar diferencias o magnitudes sin considerar la dirección.

3.5.3 Ejemplos de uso en la herramienta

1. ROUND Redondea los precios de los productos a 0 decimales

```
SELECT productName, price, ROUND(price, 0) AS roundedPrice
FROM products;
```

	productName	price	roundedPrice
►	Cemento Gris	27999.75	28000
	Ladrillo	699.99	700
	Martillo	24458.50	24459
	Pintura Blanca	44999.80	45000
	Rodillo	14999.95	15000
	Taladro	174629.95	174630
	Taladro	180000.95	180001
	Taladro	180000.95	180001

2. ABS Calcula el valor absoluto de una diferencia simulada en stock de productos

```
SELECT productName, stock - 100 AS stockDifference, ABS(stock - 100) AS
absoluteStockDifference
FROM products;
```

	productName	stockDifference	absoluteStockDifference
►	Martillo	-100	100
	Taladro	-50	50
	Cemento Gris	100	100
	Ladrillo	400	400
	Pintura Blanca	-20	20
	Rodillo	50	50
	Taladro	-50	50
	Taladro	-50	50

3.5.4 Funciones agregadas

- **COUNT:** Calcula el número total de registros en un conjunto de datos, útil para saber cuántos elementos cumplen una condición específica.
- **SUM:** Suma los valores numéricos de una columna, ideal para calcular totales en reportes financieros o de inventario.
- **AVG:** Calcula el promedio de los valores numéricos en una columna, útil para análisis estadísticos y métricas.
- **MIN:** Devuelve el valor más pequeño en una columna, útil para identificar el valor mínimo en un rango de datos.
- **MAX:** Devuelve el valor más grande en una columna, ideal para encontrar el valor máximo en un conjunto de registros.

EJEMPLOS DE USO EN LA HERRAMIENTA

-- **COUNT:** Cuenta la cantidad de productos en la tabla products

```
SELECT COUNT(*) AS totalProducts
```

```
FROM products;
```

	totalProducts
▶	8

-- **SUM:** Suma los precios de todos los productos en la tabla products

```
SELECT SUM(price) AS totalPrice
```

```
FROM products;
```

	totalPrice
▶	647789.84

-- **AVG:** Calcula el precio promedio de los productos en la tabla products

```
SELECT AVG(price) AS averagePrice
```

```
FROM products;
```

	averagePrice
▶	80973.730000

-- **MIN:** Obtiene el precio más bajo entre los productos en la tabla products

```
SELECT MIN(price) AS lowestPrice
```

```
FROM products;
```

	lowestPrice
▶	699.99

-- **MAX:** Obtiene el precio más alto entre los productos en la tabla products

```
SELECT MAX(price) AS highestPrice
```

```
FROM products;
```

	highestPrice
▶	180000.95

GROUP BY

Agrupar los registros de una consulta según los valores de una o más columnas, permitiendo aplicar funciones agregadas como COUNT, SUM, AVG, MIN, y MAX a cada grupo. Es útil para generar reportes agrupados por categorías, fechas, usuarios, etc.

EJEMPLOS DE USO EN LA HERRAMIENTA

-- **COUNT:** Cuenta cuántos productos hay en cada categoría

SELECT

categoryId,

COUNT(*) AS totalProducts

FROM products

GROUP BY categoryId;

	categoryId	totalProducts
▶	1	4
	2	2
	3	2

-- **SUM:** Suma el stock total de productos en cada categoría

SELECT

categoryId,

SUM(stock) AS totalStock

FROM products

GROUP BY categoryId;

	categoryId	totalStock
▶	1	150
	2	700
	3	230

-- **AVG:** Calcula el precio promedio de los productos en cada categoría

SELECT

categoryId,

AVG(price) AS averagePrice

FROM products

GROUP BY categoryId;

	categoryId	averagePrice
▶	1	139772.587500
	2	14349.870000
	3	29999.875000

-- MIN: Encuentra el precio más bajo de los productos en cada categoría

```
SELECT
    categoryId,
    MIN(price) AS lowestPrice
FROM products
GROUP BY categoryId;
```

	categoryId	lowestPrice
▶	1	24458.50
	2	699.99
	3	14999.95

-- MAX: Encuentra el precio más alto de los productos en cada categoría

```
SELECT
    categoryId,
    MAX(price) AS highestPrice
FROM products
GROUP BY categoryId;
```

	categoryId	highestPrice
▶	1	180000.95
	2	27999.75
	3	44999.80

HAVING:

Filtra los resultados de grupos creados con GROUP BY basándose en una condición. Es útil para aplicar criterios sobre los resultados de funciones agregadas como SUM, AVG, o COUNT.

-- HAVING SUM: Suma el stock total de productos en cada categoría y muestra solo las categorías con un stock total mayor a 200

```
SELECT
    categoryId,
    SUM(stock) AS totalStock
FROM products
```


GROUP BY categoryId

HAVING SUM(stock) > 200;

	categoryId	totalStock
▶	2	700
	3	230

-- **HAVING COUNT:** Cuenta cuántos productos hay en cada categoría y muestra solo las categorías con más de 2 productos

SELECT

categoryId,

COUNT(*) AS totalProducts

FROM products

GROUP BY categoryId

HAVING COUNT(*) > 1;

	categoryId	totalProducts
▶	1	4
	2	2
	3	2

-- **HAVING AVG:** Calcula el precio promedio de los productos en cada categoría y muestra solo las categorías con un precio promedio mayor a 50,000

SELECT

categoryId,

AVG(price) AS averagePrice

FROM products

GROUP BY categoryId

HAVING AVG(price) > 50000;

	categoryId	averagePrice
▶	1	139772.587500

-- **HAVING MIN:** Encuentra el precio más bajo de los productos en cada categoría y muestra solo las categorías donde el precio mínimo es menor a 500

SELECT

categoryId,

MIN(price) AS lowestPrice

FROM products

GROUP BY categoryId

HAVING MIN(price) < 50000;

	categoryId	lowestPrice
▶	1	24458.50
	2	699.99
	3	14999.95

-- **HAVING MAX:** Encuentra el precio más alto de los productos en cada categoría y muestra solo las categorías donde el precio máximo es mayor a 100,000

SELECT

categoryId,

MAX(price) AS highestPrice

FROM products

GROUP BY categoryId

HAVING MAX(price) > 100000;

	categoryId	highestPrice
▶	1	180000.95

3.6 Subconsultas en SQL

Las subconsultas en SQL son consultas anidadas dentro de una consulta principal que permiten realizar cálculos intermedios o proporcionar valores específicos para la consulta principal. Estas son herramientas poderosas que facilitan operaciones complejas, como filtrar datos, realizar cálculos dinámicos, o agrupar información en diferentes niveles. Se pueden utilizar en diversas partes de una consulta, como en el SELECT, el WHERE, o el FROM, para resolver problemas de manera eficiente y escalable.

3.6.1 Subconsulta en el WHERE

La subconsulta siguiente identifica el ID de la categoría llamada 'Herramientas', y la consulta principal selecciona los productos que pertenecen a esa categoría.

SELECT productName, price

FROM products

WHERE categoryId = (

SELECT categoryId

FROM categories

WHERE categoryName = 'Herramientas'

);

	productName	price
▶	Martillo	24458.50
	Taladro	174629.95
	Taladro	180000.95
	Taladro	180000.95

La subconsulta siguiente calcula el promedio de los montos totales en la tabla invoices, y la consulta principal selecciona las facturas que superan dicho promedio.

```
SELECT invoiceId, totalAmount
FROM invoices
WHERE totalAmount > (
    SELECT AVG(totalAmount)
    FROM invoices
);
```

	invoiceId	totalAmount
▶	3	150000.00
	6	180000.00
	4	200000.00
	10	300000.00
*	NULL	NULL

La subconsulta siguiente encuentra el ID del rol llamado 'Empleado' y selecciona los nombres de los usuarios asociados a este rol.

```
SELECT username
FROM users
WHERE userId IN (
    SELECT userId
    FROM userRoles
    WHERE roleId = (
        SELECT roleId
        FROM roles
        WHERE roleName = 'Empleado'
    )
);
```

	username
▶	camilot
	dianam
	miguelc

3.6.2 Subconsulta en el FROM

La subconsulta siguiente selecciona las categorías y precios de los productos. La consulta principal calcula el precio promedio por categoría.

```
SELECT categoryId, AVG(price) AS averagePrice
FROM (
    SELECT categoryId, price
    FROM products
) AS subquery
GROUP BY categoryId;
```

	categoryId	averagePrice
▶	1	139772.587500
	2	14349.870000
	3	29999.875000

La subconsulta siguiente suma las cantidades de productos vendidos por factura, y la consulta principal combina estos datos con las facturas.

```
SELECT i.invoiceId, i.totalAmount, subquery.totalProducts
FROM invoices i
JOIN (
    SELECT invoiceId, SUM(quantity) AS totalProducts
    FROM invoiceDetails
    GROUP BY invoiceId
) AS subquery
ON i.invoiceId = subquery.invoiceId;
```

	invoiceId	totalAmount	totalProducts
▶	1	60000.00	4
	2	100000.00	52
	5	120000.00	26
	3	150000.00	10
	4	200000.00	18

La subconsulta siguiente cuenta los productos en cada categoría, y la consulta principal selecciona las categorías con más de dos productos.

```
SELECT categoryName, productCount
FROM (
    SELECT c.categoryName, COUNT(p.productId) AS productCount
```

```

FROM categories c
LEFT JOIN products p ON c.categoryId = p.categoryId
GROUP BY c.categoryName
) AS subquery
WHERE productCount > 2;

```

	categoryName	productCount
▶	Herramientas	4

3.6.3 Subconsulta en el SELECT

La subconsulta siguiente calcula el precio promedio de la categoría de cada producto y lo muestra junto con el precio individual.

```

SELECT
  productName,
  price,
  (SELECT AVG(price)
   FROM products
   WHERE categoryId = p.categoryId) AS categoryAveragePrice
FROM products p;

```

	productName	price	categoryAveragePrice
▶	Martillo	24458.50	139772.587500
	Taladro	174629.95	139772.587500
	Cemento Gris	27999.75	14349.870000
	Ladrillo	699.99	14349.870000
	Pintura Blanca	44999.80	29999.875000
	Rodillo	14999.95	29999.875000
	Taladro	180000.95	139772.587500
	Taladro	180000.95	139772.587500

La subconsulta siguiente encuentra el nombre del rol asociado a cada usuario y lo muestra junto con su nombre de usuario.

```

SELECT
  username,
  (SELECT roleName
   FROM roles r
   JOIN userRoles ur ON r.roleId = ur.roleId
   WHERE ur.userId = u.userId LIMIT 1) AS roleName
FROM users u;

```

	username	roleName
▶	anam	Cliente
	camilot	Empleado
	carlosp	Proveedor
	dianam	Empleado
	luisr	Proveedor
	mariag	Proveedor
	miguelc	Empleado
	pedror	Cliente
	sofial	Cliente

La subconsulta siguiente suma las cantidades de productos vendidos en cada factura, y la consulta principal combina esta información con los datos de las facturas.

```
SELECT
    invoiceId,
    totalAmount,
    (SELECT SUM(quantity)
     FROM invoiceDetails
     WHERE invoiceId = i.invoiceId) AS totalProducts
FROM invoices i;
```

	invoiceId	totalAmount	totalProducts
▶	1	60000.00	4
	7	70000.00	NULL
	8	80000.00	NULL
	9	90000.00	NULL
	2	100000.00	52
	5	120000.00	26
	3	150000.00	10
	6	180000.00	NULL
	4	200000.00	18
	10	300000.00	NULL

3.7 Vistas

Una vista en SQL es una consulta almacenada que se puede tratar como si fuera una tabla. Proporciona una forma de presentar y consultar datos complejos de manera simplificada, reutilizando consultas frecuentes y encapsulando lógica empresarial.

-- Muestra una lista completa de productos con sus categorías asociadas.

-- Si un producto no tiene categoría asignada, el campo `categoryName` aparecerá como NULL.

```
CREATE VIEW ProductWithCategories AS
SELECT
    p.productId,
    p.productName,
    p.description,
    p.price,
    p.stock,
    c.categoryName
```

```
FROM
  products p
LEFT JOIN
  categories c ON p.categoryId = c.categoryId;
```

Hacer uso de una vista en SQL es muy similar a trabajar con una tabla normal.

-- Muestra las facturas con sus totales, el usuario que las creó y la fecha de creación.

-- Esta vista ayuda a revisar el historial de facturas y el responsable de cada una.

```
CREATE VIEW InvoiceSummary AS
SELECT
  i.invoiceId,
  i.totalAmount,
  u.username AS createdBy,
  i.createdAt
```

```
FROM
  invoices i
LEFT JOIN
  users u ON i.userId = u.userId;
```

-- Proporciona un resumen de las ventas totales y la cantidad de productos vendidos por categoría.

-- La información se agrupa por categoría y se ordena por las ventas totales de mayor a menor.

```
CREATE VIEW SalesByCategory AS
SELECT
  c.categoryName,
  SUM(id.quantity * id.price) AS totalSales,
  SUM(id.quantity) AS totalQuantity
FROM
  categories c
JOIN
  products p ON c.categoryId = p.categoryId
JOIN
  invoiceDetails id ON p.productId = id.productId
GROUP BY
  c.categoryName
ORDER BY
  totalSales DESC;
```

-- Lista los productos cuyo stock está por debajo de un nivel crítico, definido en este caso como 10.

-- Es útil para monitorear productos que necesitan ser reabastecidos.

```
CREATE VIEW LowInventory AS
SELECT
  p.productId,
  p.productName,
```

```

    p.stock,
    c.categoryName
FROM
    products p
LEFT JOIN
    categories c ON p.categoryId = c.categoryId
WHERE
    p.stock < 10;
-- Muestra un resumen combinado de entradas de material (entradas) y ventas
(salidas).

-- La información incluye el tipo (entrada o salida), el total de la transacción y la fecha.

CREATE VIEW EntriesAndExits AS
SELECT
    'Entrada' AS type,
    me.entryId AS transactionId,
    me.totalAmount,
    me.createdAt
FROM
    materialEntries me
UNION ALL
SELECT
    'Salida' AS type,
    i.invoiceId AS transactionId,
    i.totalAmount,
    i.createdAt
FROM
    invoices i
ORDER BY
    createdAt ASC;

```

3.8 Triggers

Los triggers son procedimientos almacenados que se ejecutan automáticamente en respuesta a ciertos eventos en una tabla o vista de la base de datos. Estos eventos pueden ser operaciones de INSERT, UPDATE o DELETE. Los triggers son herramientas fundamentales para garantizar la integridad de los datos, automatizar procesos y manejar tareas críticas sin intervención manual.

Un trigger puede ser configurado para ejecutarse ANTES (BEFORE) o DESPUÉS (AFTER) de que ocurra el evento.

Control de Timestamps (createdAt, updatedAt)

Es importante mantener registros de las fechas en las que los datos son creados o modificados. Para ello, se implementan triggers en las tablas relevantes:

-- Trigger para actualizar `createdAt` en la tabla invoices


```

DELIMITER $$
CREATE TRIGGER trg_invoices_created_at
BEFORE UPDATE ON invoices
FOR EACH ROW
BEGIN
    SET NEW.createdAt = CURRENT_TIMESTAMP;
END$$
DELIMITER ;
-- Trigger para actualizar `updatedAt` en la tabla products

DELIMITER $$
CREATE TRIGGER trg_products_updated_at
BEFORE UPDATE ON products
FOR EACH ROW
BEGIN
    SET NEW.updatedAt = CURRENT_TIMESTAMP;
    -- Validar que el stock no sea negativo en las actualizaciones

    IF NEW.stock < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'El stock no puede ser negativo';
    END IF;
END$$
DELIMITER ;
-- Trigger para establecer `createdAt` en la tabla products

DELIMITER $$
CREATE TRIGGER trg_products_created_at
BEFORE INSERT ON products
FOR EACH ROW
BEGIN
    SET NEW.createdAt = CURRENT_TIMESTAMP;
END$$
DELIMITER ;

```

3.9 Procedimientos Almacenados (Stored Procedures)

Un procedimiento almacenado (stored procedure) es un conjunto de instrucciones SQL predefinidas y almacenadas en la base de datos que pueden ser ejecutadas como una única unidad de trabajo. Los procedimientos almacenados son útiles para encapsular lógica de negocio, reducir redundancia en el código y mejorar la seguridad y eficiencia de las operaciones en la base de datos.

Crear una Factura

Este procedimiento permite registrar una nueva factura, junto con sus detalles, de manera transaccional.

```

DELIMITER $$
CREATE PROCEDURE createInvoice (
    IN p_userId INT,

```

```

    IN p_totalAmount DECIMAL(10,2),
    IN p_invoiceDetails JSON
)
BEGIN
    DECLARE invoiceId INT;

    -- Iniciar la transacción
    START TRANSACTION;

    -- Insertar en la tabla invoices
    INSERT INTO invoices (userId, totalAmount, createdAt)
    VALUES (p_userId, p_totalAmount, CURRENT_TIMESTAMP);

    -- Obtener el ID de la factura creada
    SET invoiceId = LAST_INSERT_ID();

    -- Insertar detalles de la factura
    INSERT INTO invoiceDetails (invoiceId, productId, quantity, price)
    SELECT invoiceId, productId, quantity, price
    FROM JSON_TABLE(
        p_invoiceDetails,
        '$[*]' COLUMNS (
            productId INT PATH '$.productId',
            quantity INT PATH '$.quantity',
            price DECIMAL(10,2) PATH '$.price'
        )
    );
    -- Confirmar la transacción
    COMMIT;
END$$
DELIMITER ;

```

Actualizar el Stock de un Producto

Este procedimiento permite actualizar el stock de un producto en función de una operación (entrada o salida).

```

DELIMITER $$
CREATE PROCEDURE updateProductStock (
    IN p_productId INT,
    IN p_quantity INT,
    IN p_operation VARCHAR(10)
)
BEGIN
    IF p_operation = 'IN' THEN
        UPDATE inventory
        SET stock = stock + p_quantity
    ELSE
        UPDATE inventory
        SET stock = stock - p_quantity
    END IF;
END$$
DELIMITER ;

```

```

        WHERE productId = p_productId;
    ELSEIF p_operation = 'OUT' THEN
        IF (SELECT stock FROM inventory WHERE productId = p_productId) < p_quantity
    THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Stock insuficiente';
    END IF;

    UPDATE inventory
    SET stock = stock - p_quantity
    WHERE productId = p_productId;
    END IF;
END$$
DELIMITER ;

```

Consultar Ventas por Usuario

Este procedimiento devuelve las ventas realizadas por un usuario en un rango de fechas.

```

DELIMITER $$
CREATE PROCEDURE getSalesByUser (
    IN p_userId INT,
    IN p_startDate DATE,
    IN p_endDate DATE
)
BEGIN
    SELECT
        s.invoiceId,
        s.createdAt,
        s.totalAmount,
        GROUP_CONCAT(CONCAT(d.productId, ': ', d.quantity) SEPARATOR ', ') AS
products
    FROM sales s
    INNER JOIN saleDetails d ON s.invoiceId = d.saleId
    WHERE s.userId = p_userId
    AND s.createdAt BETWEEN p_startDate AND p_endDate
    GROUP BY s.invoiceId;
END$$
DELIMITER ;

```

Registrar una Orden de Compra

Este procedimiento permite registrar una orden de compra y sus detalles.

```

DELIMITER $$
CREATE PROCEDURE createPurchaseOrder (
    IN p_userId INT,
    IN p_orderDetails JSON

```

```

)
BEGIN
    DECLARE purchaseOrderId INT;

    -- Iniciar la transacción
    START TRANSACTION;

    -- Insertar en la tabla purchaseOrders
    INSERT INTO purchaseOrders (userId, createdAt)
    VALUES (p_userId, CURRENT_TIMESTAMP);

    -- Obtener el ID de la orden creada
    SET purchaseOrderId = LAST_INSERT_ID();

    -- Insertar detalles de la orden de compra
    INSERT INTO purchaseOrderDetails (purchaseOrderId, productId, quantity, price)
    SELECT purchaseOrderId, productId, quantity, price
    FROM JSON_TABLE(
        p_orderDetails,
        '$[*]' COLUMNS (
            productId INT PATH '$.productId',
            quantity INT PATH '$.quantity',
            price DECIMAL(10,2) PATH '$.price'
        )
    );

    -- Confirmar la transacción
    COMMIT;
END$$
DELIMITER ;

```

3.10 Encriptación de Contraseñas

La seguridad de los datos sensibles, como las contraseñas, es un aspecto crítico en cualquier sistema. En este proyecto, implementamos un mecanismo de encriptación de contraseñas para proteger los datos almacenados en la base de datos. Esto garantiza que, incluso si la base de datos es comprometida, las contraseñas no serán legibles, mejorando la seguridad general del sistema.

Proceso de Encriptación de Contraseñas

La seguridad de las contraseñas es un componente crítico en cualquier sistema que maneje datos sensibles de usuarios. Para garantizar la protección de las contraseñas almacenadas en la base de datos, hemos implementado técnicas de encriptación utilizando el algoritmo SHA-256 a través de la función SHA2 de MySQL. Este enfoque asegura que las contraseñas nunca se almacenen en texto plano, dificultando el acceso no autorizado incluso si la base de datos se ve comprometida.

Tal como podemos ver en la imagen la columna 'passwordHash' es de tipo texto.

	userId	username	passwordHash	email	personId	status	createdAt	updatedAt
▶	1	carlosp	hashedpass1	carlos.perez@ejemplo.com	1	active	2024-12-05 19:28:56	2024-12-05 19:28:56
	2	mariag	hashedpass2	maria.gomez@ejemplo.com	2	active	2024-12-05 19:28:56	2024-12-05 19:28:56
	3	luisr	hashedpass3	luis.rojas@ejemplo.com	3	active	2024-12-05 19:28:56	2024-12-05 19:28:56
	4	anam	hashedpass4	ana.martinez@ejemplo.com	4	active	2024-12-05 19:28:56	2024-12-05 19:28:56
	5	pedror	hashedpass5	pedro.ramirez@ejemplo.com	5	active	2024-12-05 19:28:56	2024-12-05 19:28:56
	6	sofial	hashedpass6	sofia.lopez@ejemplo.com	6	active	2024-12-05 19:28:56	2024-12-05 19:28:56
	7	camilot	hashedpass7	camilo.torres@ejemplo.com	7	active	2024-12-05 19:28:56	2024-12-05 19:28:56
	8	dianam	hashedpass8	diana.munoz@ejemplo.com	8	active	2024-12-05 19:28:56	2024-12-05 19:28:56
	9	miguelc	hashedpass9	miguel.castro@ejemplo.com	9	active	2024-12-05 19:28:56	2024-12-05 19:28:56
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Para realizar la encriptación realizamos los siguientes pasos:

```

1      -- Modificar la Columna password para Soportar Datos Cifrados
2
3  •   ALTER TABLE users MODIFY COLUMN `password` CHAR(64);
4
5      -- Encriptar la Contraseña de un Usuario
6
7  •   UPDATE users
8
9      SET `passwordHash` = SHA2('securepassword', 256)
10     WHERE `userId` = 1;
11
12     -- Verificar la Contraseña Encriptada
13
14  •   SELECT `userId`
15     FROM users
16     WHERE `userId` = 1 AND `passwordHash` = SHA2('securepassword', 256);
17

```

Si revisamos nuevamente la consulta nos da como resultado lo siguiente:

	userId	username	passwordHash	email	personId	status	createdAt	updatedAt
▶	1	carlosp	BLOB	carlos.perez@ejemplo.com	1	active	2024-12-05 19:28:56	2024-12-06 00:40:33
	2	mariag	BLOB	maria.gomez@ejemplo.com	2	active	2024-12-05 19:28:56	2024-12-05 19:28:56
	3	luisr	BLOB	luis.rojas@ejemplo.com	3	active	2024-12-05 19:28:56	2024-12-05 19:28:56
	4	anam	BLOB	ana.martinez@ejemplo.com	4	active	2024-12-05 19:28:56	2024-12-05 19:28:56
	5	pedror	BLOB	pedro.ramirez@ejemplo.com	5	active	2024-12-05 19:28:56	2024-12-05 19:28:56
	6	sofial	BLOB	sofia.lopez@ejemplo.com	6	active	2024-12-05 19:28:56	2024-12-05 19:28:56
	7	camilot	BLOB	camilo.torres@ejemplo.com	7	active	2024-12-05 19:28:56	2024-12-05 19:28:56
	8	dianam	BLOB	diana.munoz@ejemplo.com	8	active	2024-12-05 19:28:56	2024-12-05 19:28:56
	9	miguelc	BLOB	miguel.castro@ejemplo.com	9	active	2024-12-05 19:28:56	2024-12-05 19:28:56
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Como podemos ver la columna passwordHas se encuentra encriptada.

3.11 Respaldo de Bases de Datos

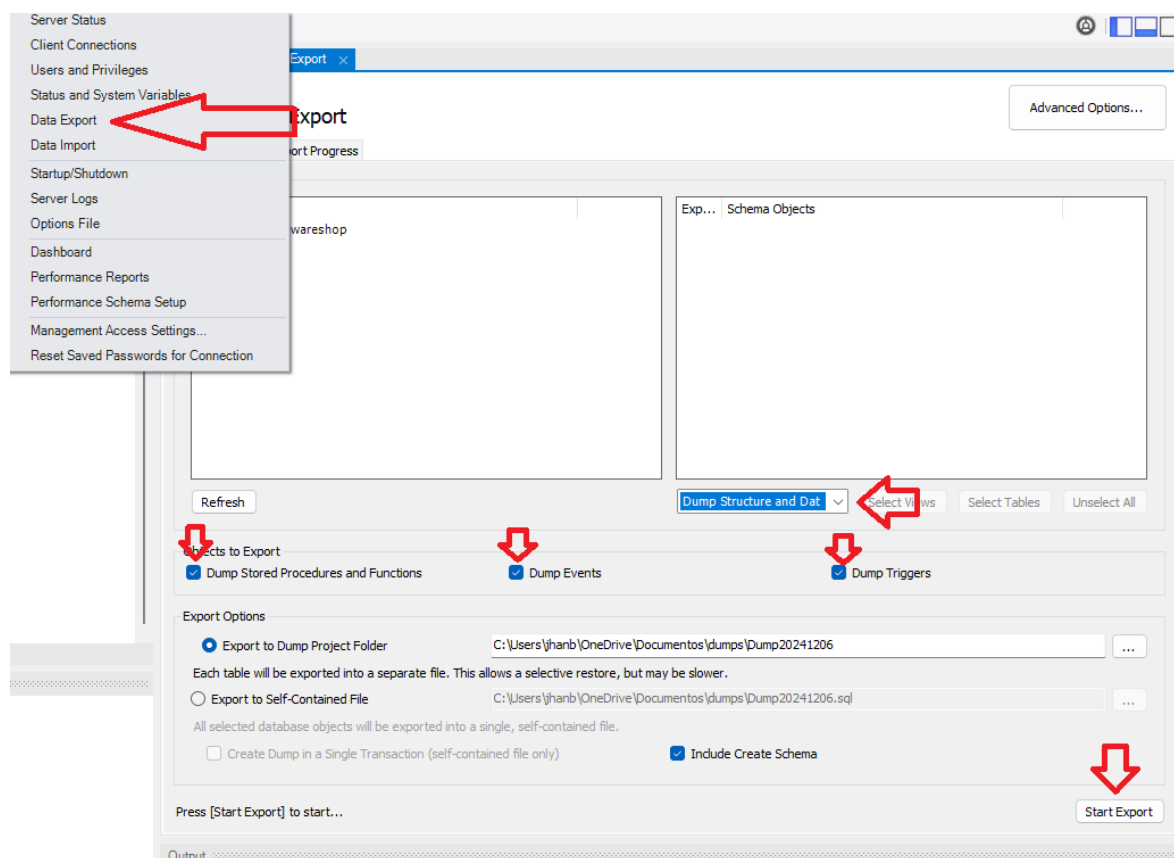
El respaldo o backup de una base de datos es una tarea fundamental para garantizar la disponibilidad y recuperación de información en caso de fallos del sistema, pérdida de datos o ataques malintencionados. Implementar un sistema de respaldo periódico no solo protege la integridad de los datos, sino que también asegura la continuidad de las operaciones del negocio ante cualquier eventualidad.

En este proyecto, se utilizó MySQL Workbench como herramienta para realizar respaldos totales de la base de datos mediante la funcionalidad de Data Export. Esta opción permite generar copias completas de la base de datos en formato SQL, que pueden ser restauradas fácilmente en cualquier momento.

MySQL Workbench es una herramienta gráfica que facilita la administración y mantenimiento de bases de datos MySQL. La funcionalidad de Data Export permite:

1. Exportar el esquema y los datos de todas las tablas de la base de datos.
2. Generar un archivo .sql que incluye las instrucciones necesarias para recrear tanto la estructura como el contenido de la base de datos.
3. Elegir entre respaldos parciales (solo ciertas tablas) o totales (toda la base de datos).

En este proyecto, se opta por realizar respaldos totales para garantizar que todos los datos y la estructura de la base de datos estén protegidos.



4 Conclusiones

- El desarrollo de este proyecto de base de datos para la ferretería ficticia **hardwareshop** ha permitido implementar un sistema sólido, eficiente y alineado con las buenas prácticas en el diseño y gestión de bases de datos. A lo largo del proyecto, se abordaron múltiples aspectos técnicos que contribuyen tanto a la funcionalidad del sistema como a la seguridad y escalabilidad de la base de datos.
- Se logró una base de datos altamente normalizada, con relaciones claras y bien definidas, lo que minimiza la redundancia de datos y mejora la consistencia.
- El uso de triggers en la base de datos aporta un alto grado de automatización, consistencia y seguridad a las operaciones realizadas. Implementar triggers correctamente asegura que las reglas de negocio se cumplan y que la base de datos mantenga un estado íntegro frente a inserciones, actualizaciones y eliminaciones. En este proyecto, los triggers aplican validaciones, cálculos automáticos y rastrean cambios críticos, proporcionando un sistema sólido y confiable.
- La implementación de la encriptación de contraseñas asegura la protección de los datos sensibles de los usuarios, cumpliendo con estándares básicos de seguridad.
- Se definió una estrategia de respaldo completa utilizando la funcionalidad de Data Export de MySQL Workbench, garantizando la protección de la información y la posibilidad de recuperación ante fallos o pérdida de datos.
- El diseño modular y la estandarización de nombres permiten la fácil incorporación de nuevas funcionalidades sin comprometer la estructura actual.

5 Recomendaciones

Al finalizar el desarrollo del proyecto de la base de datos para hardwareshop, se identificaron oportunidades de mejora y buenas prácticas que podrían implementarse para optimizar el sistema y garantizar su sostenibilidad a largo plazo. A continuación, se presentan recomendaciones técnicas y organizacionales:

- **Implementar Salting en Contraseñas:** Aunque se utiliza encriptación con SHA-256, agregar un "salt" único a las contraseñas aumentará significativamente la resistencia a ataques de diccionario y fuerza bruta. Utilizar algoritmos como bcrypt o argon2, que ya incorporan salting y ofrecen mayor seguridad.
- **Control de Acceso:** Configurar roles de usuario con privilegios mínimos necesarios para reducir el riesgo de acceso no autorizado. Establecer conexiones seguras (SSL/TLS) para proteger la comunicación entre el servidor y los clientes.
- **Auditoría de Cambios:** Implementar una tabla de auditoría para registrar modificaciones críticas en los datos (usuarios, inventarios, ventas), asegurando trazabilidad y responsabilidad.
- **Pruebas Regulares de Restauración:** Realizar pruebas periódicas de restauración de los respaldos para asegurar que los archivos sean funcionales y completos.
- **Monitoreo Continuo:** Implementar herramientas de monitoreo para identificar problemas de rendimiento, espacio en disco o posibles fallos antes de que afecten el sistema.